

Algoritmos de Alto Desempenho

Relatório do Segundo Projeto

Luis Sousa: n^omec 108583
Simão Almeida: n^omec 113085



universidade
de aveiro

luisbfsousa@ua.pt
spsa@ua.pt

Universidade de Aveiro
Departamento de Electrónica, Telecomunicações e Informática
2026

Conteúdo

1	Introdução	2
2	Arquitetura da Solução	3
2.1	Alinhamento e Gestão de “Missing Bits”	3
2.2	Geração de Máscaras e Extração do MSB	3
2.3	Lógica de Reconstrução	4
2.3.1	Implementação <i>Bit-Serial</i>	4
3	Implementação	5
3.1	Módulo Principal (<code>bfe.vhd</code>)	5
3.2	Comparador estrutural (<code>comparator_n.vhd</code>)	5
3.3	Testbench e Verificação (<code>textttbfe_tb.c</code>)	5
4	Resultados de Simulação	6
4.1	Validação Funcional Geral	6
4.2	Verificação dos “Missing Bits”	6
4.3	Análise Temporal e Atrasos	7
5	Conclusão	8
6	Apêndice	9
6.1	<code>bfe.vhd</code>	9
6.2	<code>comparator_n.vhd</code>	10
6.3	Função <code>do_one</code> do <code>bfe_tb.c</code> (Para testes)	11

1 Introdução

Este projeto tem como objetivo o desenvolvimento e simulação da lógica combinatória subjacente à instrução de extração de campos de bits (Bit-Field Extract - **bfe**) de um processador fictício, em VHDL.

A instrução **bfe** opera sobre um registo de origem (**src**), extraíndo uma sequência de bits definida por uma posição inicial (**start**) e um tamanho (**size**), depositando o resultado num registo de destino (**dst**). Foram desenvolvidas duas variantes da extensão: a variante **.u** (unsigned), que realiza a extensão a zeros, e a variante **.s** (signed), que realiza a extensão de sinal do campo extraído.

Alem disso, a solução apresentada trata dos bits em falta de forma dinâmica, preenchendo com zeros na variante **.u** e com o bit de sinal na variante **.v**. Também foi implementada uma versão do comparador de N bits, simulando atrasos de propagação realistas.

2 Arquitetura da Solução

A solução desenvolvida para a instrução `bfe` (Bit-Field Extract) baseia-se numa arquitetura modular e combinatória. O fluxo de dados foi desenhado para maximizar a reutilização dos componentes fornecidos como o `barrel_shift_right` e `comparator_n`, introduzindo lógica de controlo eficiente para gerir as máscaras de bits e a extensão de sinal.

A arquitetura pode ser dividida em três blocos funcionais principais: o Alinhamento (Shifter), a Geração de Máscaras e a Lógica de Reconstrução/Extensão.

2.1 Alinhamento e Gestão de “Missing Bits”

O primeiro passo do procedimento consiste em alinhar o campo de bits de interesse para a posição menos significativa (LSB). Para tal, foi instanciado o componente `barrel_shift_right`.

Também foi implementada a gestão dos bits em falta (*missing bits*) diretamente no deslocamento. O sinal de controlo `variant` da instrução foi ligado diretamente á porta `missing` do *barrel shifter*:

- **Variante .u (`variant='0'`):** O *shifter* opera em modo lógico, preenchendo os bits introduzidos à esquerda com 0.
- **Variante .s (`variant='1'`):** O *shifter* opera em modo aritmético, preenchendo os bits em falta com o bit mais significativo (MSB) do registo `src`.

Esta abordagem permite simplificar a lógica subsequente, pois garante que os bits que “caem fora” do registo original são tratados corretamente na fonte.

2.2 Geração de Máscaras e Extração do MSB

Paralelamente ao deslocamento, o circuito necessita de identificar quais os bits do registo deslocado pertencem efetivamente ao campo de tamanho `size`.

Para isto, foi implementada uma estrutura `generate` que instancia N comparadores (`comparator_n`), onde N é o número de bits do registo (16). Cada comparador avalia a posição exata do bit face ao tamanho do corpo:

- **Máscara de Campo (`msfb_mask`):** Identifica a posição exata do bit mais significativo do campo extraído.
- **Máscara de Limpeza (`mask`):** Identifica todos os bits que estão acima do tamanho do campo.

O bit de sinal do campo extraído é obtido através de uma operação de redução OR sobre o vetor resultante da operação `shifted_src AND msfb_mask`.

2.3 Lógica de Reconstrução

Esta fase combina os sinais anteriores para produzir o resultado **dst**. São gerados dois sinais intermédios:

1. **zero_extended**: Resulta do **shifted_src** filtrado pela **mask** invertida, garantindo que todos os bits acima de **size** são zero.
2. **sign_extended**: Utiliza a mesma base, mas para os bits onde a **mask** é ativa (fora do campo), preenche com o bit de sinal extraído anteriormente.

A saída final **dst** é selecionada através de um multiplexer controlado pelo sinal **variant**.

2.3.1 Implementação *Bit-Serial*

Nesta implementação, foi desenvolvida uma arquitetura estrutural em cadeia.

Aqui, a decisão de comparação propaga-se do bit menos significativo (LSB) para o mais significativo (MSB). Cada bit recebe o resultado da comparação anterior e decide o novo estado:

- Se os bits atuais forem iguais, o estado anterior é propagado.
- Se forem diferentes, o bit atual define o resultado.

Foi introduzido um atraso de 5 ps por estágio. Esta modelação temporal permite simular com realismo o tempo de propagação físico do sinal através das portas lógicas, o que é fundamental para a análise correta do período mínimo de relógio do circuito global.

3 Implementação

A implementação do circuito foi realizada em VHDL, tirando partido de funcionalidades modernas da linguagem para tornar o código mais conciso e legível. Foram alterados dois ficheiros principais de descrição de hardware (`bfe.vhd` e `comparator_n.vhd`).

3.1 Módulo Principal (`bfe.vhd`)

O ficheiro `bfe.vhd` contém a integração de todos os subcomponentes. A implementação destaca-se pelos seguintes pontos:

- **Instanciação do *Barrel Shifter*:** O componente `barrel_shift_right` foi instanciado mapeando a porta `missing` diretamente ao sinal de entrada `variant`.

```
missing => variant -- '0' para shift lógico (.u), '1' para aritmético (.s)
```

Esta ligação permite que o deslocamento trate automaticamente do preenchimento de bits.

- **Geração de Máscaras com `generate`:** Foi utilizado um ciclo `for ... generate` para instanciar 16 comparadores em paralelo. Cada iteração compara o índice fixo `i` com o sinal `size`, produzindo os sinais individuais que compõem os vetores `msfb_mask` e `mask`.
- **Redução da lógica:** Para extrair o bit de sinal do campo (`msb_bit`), utilizou-se o operador de redução unário `or`, que aplica a operação a todos os bits resultantes da operação `and`.

```
msb_bit <= or (shifted_src and msfb_mask);
```

3.2 Comparador estrutural (`comparator_n.vhd`)

O comparador foi implementado através de uma arquitetura *bit-serial*. A implementação baseia-se num *array* de sinais intermédios (`c_lt`, `c_eq`, `c_gt`) que propagam o estado da comparação do bit menos significativo para o mais significativo. Foi modelado um atraso de transporte para simular a latência real das portas lógicas:

```
lt <= transport c_lt(N) after 5 ps * N;
```

Este atraso acumulado (20 ps para 4 bits) é essencial para a análise temporal subsequente.

3.3 Testbench e Verificação (`textttbfe_tb.c`)

Para validar corretamente a funcionalidade avançada de *missing bits*, foi necessário alterar a lógica de referência do ficheiro `bfe_tb.c`, de modo a que este aceite um comportamento híbrido ao invés de aceitar apenas um comportamento exclusivo usando a diretiva de pré-processamento `if ... else`.

4 Resultados de Simulação

A validação do circuito foi realizada utilizando o simulador GHDL e o visualizador de formas de onda GTKWave. O testbench utilizado foi gerado pelo código C modificado (bfe_tb.c) para contemplar o comportamento avançado de extensão de sinal nos bits em falta.

Abaixo apresentam-se os resultados que comprovam o funcionamento funcional e temporal da solução.

4.1 Validação Funcional Geral

A Figura 1 apresenta uma visão geral da simulação. É possível observar a correspondência entre o sinal de saída do circuito (`s_dst`) e o sinal de referência esperado (`s_expected`).

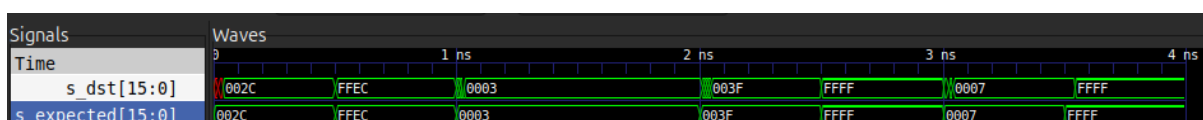


Figura 1: Visão geral da simulação

Nota-se que, após o período de estabilização inicial (onde o sinal aparece como uuuu), o circuito responde corretamente a todas as transições de estímulo, validando a lógica de deslocamento e geração de máscaras.

4.2 Verificação dos “Missing Bits”

Para validar o tratamento inteligente de bits fora dos limites (missing bits), analisou-se o caso de teste específico onde `src = FFFF`, `size = 4` e `start = 13`. Neste cenário, o campo extraído ultrapassa o limite do registo de origem.

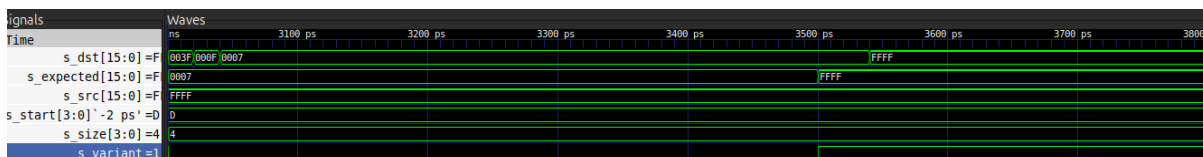


Figura 2: Detalhe da transição da variante Unsigned (.u) para Signed (.s) com bits em falta.

- **Variante .u (início):** O resultado é 0007 (binário 0...0111). O circuito preencheu o bit em falta (bit 16) com '0', resultando num valor positivo pequeno.
- **Variante .s (final):** Quando o sinal `variant` muda para '1', o resultado altera-se para FFFF (binário 1...1111). O circuito detetou que o MSB do `src` era '1' e utilizou-o para preencher o bit em falta, realizando corretamente a extensão de sinal aritmética.

Esta transição de 0007 para FFFF comprova que a ligação do sinal `variant` à porta `missing` do *barrel shifter* está a funcionar conforme planeado.

4.3 Análise Temporal e Atrasos

A implementação estrutural do comparador e do *shifter* introduz atrasos de propagação realistas. A Figura 3 apresenta um zoom numa transição para evidenciar a latência do circuito.

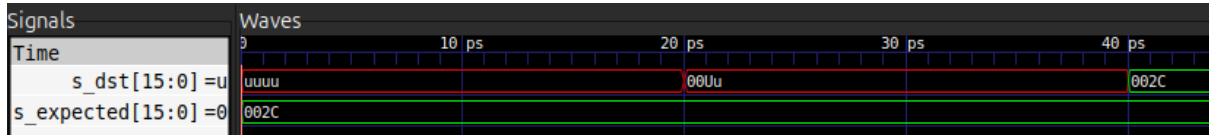


Figura 3: Visualização do atraso de propagação

Observa-se um desfasamento temporal entre a atualização do sinal esperado e a estabilização do sinal de saída. A análise da arquitetura revela que o Caminho Crítico (o percurso mais lento) é determinado pelo *Barrel Shifter*, visto que este e o comparador operam em paralelo:

- **Barrel Shifter:** $4 \text{ fases} * 10\text{ps/fase} = 40\text{ps}$
- **Comparador:** $4 \text{ bits} * 5\text{ps/bit} = 20\text{ps}$

Como os blocos operam simultaneamente, o tempo de estabilização final é ditado pelo componente mais lento ($\max(40,20)$), resultando num atraso total de 40 ps. Este valor define o limite máximo teórico para a frequência de operação do circuito.

5 Conclusão

Este projeto permitiu a implementação e validação bem-sucedida da lógica combinatória para a instrução Bit-Field Extract (**bfe**), cumprindo todos os requisitos funcionais e temporais propostos.

A solução desenvolvida destaca-se não apenas pela implementação correta das variantes *unsigned* (.u) e *signed* (.s), mas também pela integração da arquitetura *bit-serial*:

1. Gere dinamicamente os bits em falta, aplicando corretamente a extensão de sinal ou de zeros conforme a variante da instrução.
2. Simula os atrasos de propagação física.

A validação experimental, realizada através de um testbench em C adaptado e simulação em GHDL, confirmou a correção lógica do circuito para diversos casos de teste, incluindo situações de limite. Adicionalmente, a análise das formas de onda permitiu determinar o caminho crítico do circuito, fixando o período mínimo de relógio funcional em aproximadamente 40 ps.

6 Apêndice

6.1 bfe.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity bfe is
  generic
  (
    DATA_BITS_LOG2 : integer range 2 to 6 := 4           -- use 4 by default
  );
  port
  (
    dst      : out std_logic_vector(2**DATA_BITS_LOG2-1 downto 0); -- 15 downto 0
    src      : in  std_logic_vector(2**DATA_BITS_LOG2-1 downto 0); -- 15 downto 0
    size     : in  std_logic_vector( DATA_BITS_LOG2-1 downto 0); -- 3 downto 0
    start    : in  std_logic_vector( DATA_BITS_LOG2-1 downto 0); -- 3 downto 0
    variant  : in  std_logic                                     -- '0' for .u and '
  );
end bfe;

architecture structural of bfe is
  constant N_BITS : integer := 2**DATA_BITS_LOG2;

  -- internal signals
  signal shifted_src      : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
  signal msfb_mask        : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
  signal mask             : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
  signal msb_bit          : std_logic;
  signal sign_extended    : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
  signal zero_extended    : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
  signal msfb_mask_eq     : std_logic_vector(2**DATA_BITS_LOG2-1 downto 0);
begin
  barrel_shifter : entity work.barrel_shift_right(behavioral)
    generic map
    (
      DATA_BITS_LOG2 => DATA_BITS_LOG2
    )
    port map
    (
      data_in  => src,
      data_out => shifted_src,
      shift    => start,
      missing  => variant
    );
end;
```

```

mask_gen : for i in 0 to 2**DATA_BITS_LOG2-1 generate
  comp : entity work.comparator_n(structural)
    generic map
      (
        N => DATA_BITS_LOG2
      )
    port map
      (
        a => std_logic_vector(to_unsigned(i, DATA_BITS_LOG2)),
        b => size,
        lt => open,
        eq => msfb_mask_eq(i),
        gt => mask(i)
      );
end generate;

msfb_mask <= msfb_mask_eq;

msb_bit <= or (shifted_src and msfb_mask);

zero_extended <= shifted_src and not mask;

sign_ext_gen : for i in 0 to 2**DATA_BITS_LOG2-1 generate
  sign_extended(i) <= shifted_src(i) when mask(i) = '0' else msb_bit;
end generate;

dst <= sign_extended when variant = '1' else zero_extended;

end structural;

```

6.2 comparator_n.vhd

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity comparator_n is
  generic
    (
      N : positive
    );
  port
    (
      a : in  std_logic_vector(N-1 downto 0);
      b : in  std_logic_vector(N-1 downto 0);
      lt : out std_logic; -- '1' if a<b, '0' otherwise
      eq : out std_logic; -- '1' if a=b, '0' otherwise
    );
end entity comparator_n;

```

```

        gt : out std_logic -- '1' if a>b, '0' otherwise
    );
end comparator_n;

```

```

architecture structural of comparator_n is
    signal c_lt : std_logic_vector(0 to N);
    signal c_eq : std_logic_vector(0 to N);
    signal c_gt : std_logic_vector(0 to N);
begin
    c_lt(0) <= '0';
    c_eq(0) <= '1';
    c_gt(0) <= '0';
    chain_gen: for i in 0 to N-1 generate
        process(a(i), b(i), c_lt(i), c_eq(i), c_gt(i))
        begin
            if a(i) = b(i) then
                c_lt(i+1) <= c_lt(i);
                c_eq(i+1) <= c_eq(i);
                c_gt(i+1) <= c_gt(i);
            elsif a(i) = '1' then
                c_lt(i+1) <= '0';
                c_eq(i+1) <= '0';
                c_gt(i+1) <= '1';
            else
                c_lt(i+1) <= '1';
                c_eq(i+1) <= '0';
                c_gt(i+1) <= '0';
            end if;
        end process;
    end generate;

    lt <= transport c_lt(N) after 5 ps * N;
    eq <= transport c_eq(N) after 5 ps * N;
    gt <= transport c_gt(N) after 5 ps * N;

end structural;

```

6.3 Função do_one do bfe_tb.c (Para testes)

```

static void do_one(unsigned long src,int size,int start)
{
    unsigned long dst0, dst1;
    unsigned long src_u, src_s;

    // Preparar versão com Zero-Extend
    src_u = src << (64 - n_bits);
    src_u = src_u >> (64 - n_bits);

```

```

// Preparar versão com Sign-Extend (para variante)
src_s = src << (64 - n_bits);
src_s = (unsigned long)((signed long)src_s >> (64 - n_bits));

// Variante .u (usa src_u - missing bits a 0)
dst0 = src_u >> start;
dst0 = dst0 << (63 - size);
dst0 = dst0 >> (63 - size);

// Variante .s (usa src_s - missing bits = MSB)
dst1 = src_s >> start;
dst1 = dst1 << (63 - size);
dst1 = (unsigned long)((signed long)dst1 >> (63 - size));

// output test data
printf(
    "    --- new test case"
    "    s_src      <= \"%s\";"
    "    s_size     <= \"%s\"; -- %d"
    "    s_start    <= \"%s\"; -- %d"
    "    s_variant  <= '0';"
    "    s_expected <= \"%s\";"
    "    wait for 499 ps;"
    "    assert s_dst = s_expected report \"bad dst\" severity note;"
    "    wait for 1 ps;"
    "    s_variant  <= '1';"
    "    s_expected <= \"%s\";"
    "    wait for 499 ps;"
    "    assert s_dst = s_expected report \"bad dst\" severity note;"
    "    wait for 1 ps;"
    to_binary(src,n_bits),
    to_binary(size,n_bits_log2),size,
    to_binary(start,n_bits_log2),start,
    to_binary(dst0,n_bits),
    to_binary(dst1,n_bits)
);
}

```

```

"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n
"\n

```